INTERNSHIP REPORT PRESENTATION

MERN Full Stack Development Virtual Internship

AICTE – EduSkills Virtual Internship Program | Aug – Oct 2026

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

Presented by

SANJIV SINGH

Roll No: 2410031649

B.Tech CSE | Batch 2024–2028

AICTE ID: STU69ebb4588f93c1777054808

Internship at a Glance

Student	Sanjiv Singh
Branch	B.Tech (Computer Engineering)
Institute	IILM University, Greater Noida
Domain	MERN Full Stack Development + Project
Duration	August 2026 – October 2026
Reference No.	C18Y2026M101533528

8
Weeks

100%
Virtual / Online

5
Project Problems

1
Certificate Earned

Organization Profile

Organization

EduSkills Academy (EduSkills Foundation)

Domain

EdTech & Skill Development

Services

Virtual Internships

Vision

To empower students with industry-relevant skills

Offered under the patronage of AICTE and the National Internship Portal, Government of India.

Internship Overview & Evaluation

Mode	Online / Virtual
Duration	8 Weeks
Platform	EduSkills LMS
Role	Intern

How we were evaluated

- Week-wise structured learning plan
- Weekly assessments to track progress
- Project documentation and assignments
- Final Assessment Test in the last week
- Final grade from assessments, projects and final test

8-Week Learning Journey

Week 1

Web Setup & Core JavaScript

Week 2

React Fundamentals & Hooks

Week 3

Context API, Redux Toolkit & Router

Week 4

TailwindCSS & React SPA

Week 5

Node.js, Express & MongoDB

Week 6

REST APIs & JWT Auth

Week 7

Full-Stack Integration & DevOps

Week 8

Capstone: E-commerce Platform

Frontend (Weeks 1–4) → Backend (Weeks 5–6) → Integration & Capstone (Weeks 7–8)

Project: Text Search Indexer

Problem Statement

A system to process large volumes of text and build an efficient index for rapid searching. Text is tokenized and normalized, then stored in an inverted index mapping each word to the lines it appears on. It supports single-keyword and multi-keyword AND queries, and reports matching lines with keyword occurrence counts.

Objectives



Design and implement a solution for the given problem



Use modern tools and algorithms



Optimize performance and code quality

Methodology

1

Tokenize

Split raw text into words

2

Normalize

Lowercase, strip special characters

3

Build Index

Map each word to its line numbers

4

Search

Single and multi-keyword AND lookup

5

Summarize

Occurrence counts per keyword

Built sequentially and modularly, each stage feeding the next.

Tools & Technologies

Language

JavaScript (Node.js)

Core Features

String manipulation, Array methods, Object literals

Data Structures

Hash maps (objects) and arrays

Concepts

Tokenization, Normalization, Inverted Indexing, Information Retrieval

Implementation: Five Problems Solved

1	**Tokenize & Normalize Text**	Lowercase, strip punctuation, split into clean word tokens.
2	**Build Word-to-Line Index**	Map every unique word to the line numbers where it occurs.
3	**Single Keyword Search**	Look up a keyword directly in the index for matching lines.
4	**Multi-Keyword AND Search**	Intersect line lists so every keyword appears on the line.
5	**Occurrence Count Summary**	Report how often each queried keyword appears in results.

Code: Tokenize & Normalize

```
const lines = input.split('\n');
const results = [];
for (const line of lines) {
  let result;
  if (line.trim() === '') {
    result = { "words": [] };
  } else {
    const words = line
      .toLowerCase()
      .replace(/[^a-z0-9\s]/g, '')
      .trim()
      .split(/\s+/)
      .filter(w => w !== '');
    result = { "words": words };
  }
  results.push(result);
}
console.log(JSON.stringify(results));
```

What it does

- Converts each line to lowercase
- Removes non-alphanumeric characters with a regex
- Splits on whitespace and drops empty tokens
- Handles empty lines safely

Code: Index & AND Search

Inverted index

```
let index = {};
for (let i = 0; i < lines.length; i++) {
  if (lines[i].trim() === '') continue;
  const words = lines[i].toLowerCase()
    .replace(/[^a-z0-9\s]/g, '')
    .split(/\s+/);
  words.forEach(word => {
    if (!index[word]) index[word] = [];
    if (!index[word].includes(i))
      index[word].push(i);
  });
}
```

Multi-keyword AND

```
let found = index[keywords[0]] || [];
for (let i = 1; i < keywords.length; i++) {
  const next = index[keywords[i]] || [];
  found = found.filter(
    line => next.includes(line)
  );
}
results.push(found);
```

Example: index = {"hello":[0,1], "world":[0,1], "test":[1]}. Query "hello test" returns line [1]; "hello" returns [0, 1].

Learning Outcomes

Text processing	Tokenization and normalization techniques
Inverted indexing	How indexes enable efficient text search
Search algorithms	Single and multi-keyword query design
Data structures	JavaScript objects and arrays for indexing
Problem solving	Complex string operations and search logic

Challenges Faced & Solutions

Challenge	Solution
Case and punctuation in matching	Multi-step normalization: lowercase plus regex to remove unwanted characters.
Efficient multi-keyword AND search	Intersected the line-number lists retrieved from the index for each keyword.
Accurate occurrence counts	Re-processed only the matched lines to count keywords, since the index tracks presence, not counts.

Conclusion & Future Scope

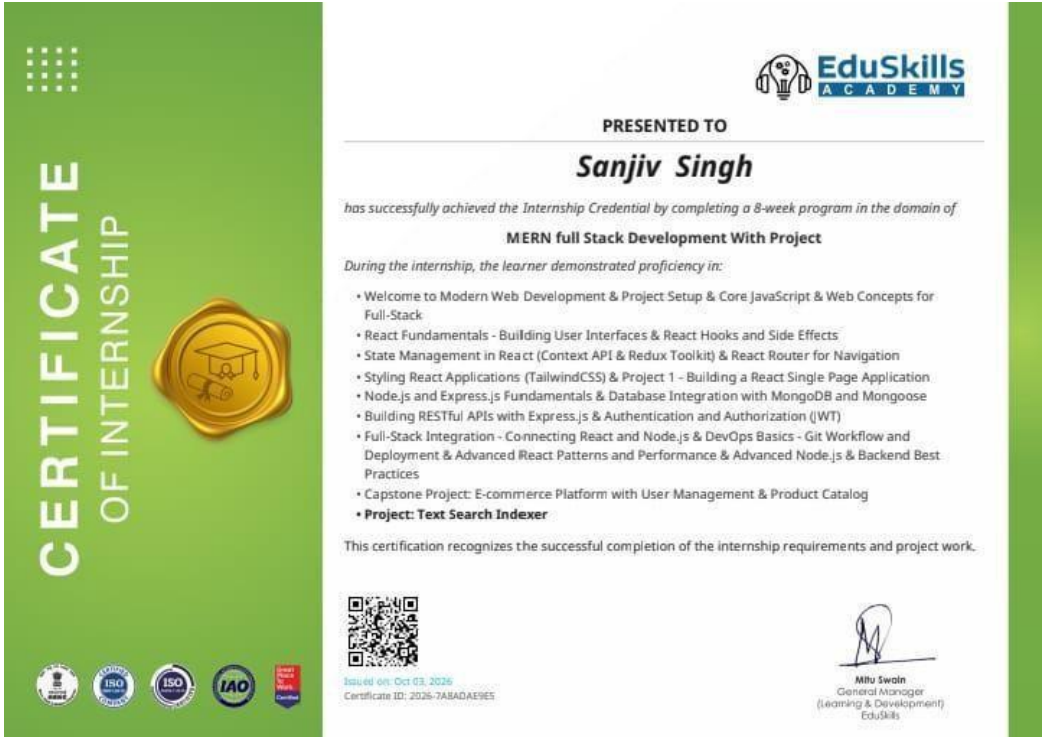
Conclusion

The Text Search Indexer delivers an efficient approach to text processing and retrieval. Normalization, inverted indexing, single and multi-keyword search and occurrence summaries show core data structure and algorithm principles applied to real information-access problems.

Future Scope

- OR and NOT search operators
- Fuzzy search for misspellings and variations
- Search across multiple documents or files concurrently

Internship Completion Certificate



Issued by
EduSkills Academy

Issued on
03 October 2026

Certificate ID
2026-7A8ADAE9E5

Project
Text Search Indexer

Thank You

Sanjiv Singh | Roll No. 2410031649 | B.Tech CSE, IILM University, Greater Noida

Thanks to AICTE, EduSkills, my industry mentor and the Department of Computer Science.